

# Source spoofing and reflection

Follow-along: play the attacker, then the defender

## Overview

### Definition: Source-address spoofing

Forging the source-address field of an outgoing packet so it names an address the sender does not own. UDP carries no handshake, so nothing stops a host from claiming any source; the receiver, and any reply it sends, treats the forged address as genuine.

### Definition: Reflection

Bouncing traffic off a third-party server instead of sending it to the target directly. The attacker sends a request with the victim's address forged as the source; the server answers the source it was handed, so its reply lands on the victim, which never asked.

**Source-address spoofing** and **reflection** turn ordinary public servers into a weapon aimed at someone else. The attacker never contacts the victim directly: it sends a small request to a server that answers over UDP, and forges the request's source so the answer is delivered to the victim. Because a UDP server trusts the source field and replies to whoever the packet claims to be from, the server does the delivering, and its own address, not the attacker's, is what the victim sees.

### Definition: Amplification

The reply a server returns being larger than the request that triggered it. The ratio of reply size to request size is the amplification factor: a factor of 50 means one byte sent draws fifty bytes delivered to the victim.

The second ingredient is **amplification**. Some replies are many times larger than the request that draws them, so one attacker on a weak uplink can aim a large, hard-to-trace flood at a target it never touches. This lab builds that mechanism from two everyday services, DNS and NTP, measures how far each one amplifies, then closes the single hole that makes it possible: a router that forwards packets without checking whether a source address could legitimately have arrived where it did.

## Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one lab per row. Select **Source spoofing and reflection** to open its

actions:

- **Spawn** creates this lab's router, switch, attacker, victim, and two reflectors, and wires them. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the addressing, the router's filter state, and the oracle: does a forged-source query still reach each reflector (REACHES) or not (BLOCKED).
- **Shell command for a node** asks which node (attacker, router, victim, dns, ntp) and prints the `docker exec -it ...` line to paste into your own terminal; each step below names the node it runs on.
- **Reset to baseline** stops the attack and drops your filter, returning the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

## The setup

This lab is Layer 3, so it is not one flat LAN. A single router forwards between two subnets, and everything the attack and the defence do happens across that router:

| Role          | IP                                   | Subnet                | Container           |
|---------------|--------------------------------------|-----------------------|---------------------|
| attacker      | 102.0.0.10                           | customer 102.0.0.0/24 | 102_L3_LAB_attacker |
| router        | 102.0.0.1 (cust)<br>102.1.0.1 (core) | both                  | 102_L3_LAB_router   |
| victim        | 102.1.0.20                           | core 102.1.0.0/24     | 102_L3_LAB_victim   |
| DNS reflector | 102.1.0.53                           | core                  | 102_L3_LAB_dns      |
| NTP reflector | 102.1.0.123                          | core                  | 102_L3_LAB_ntp      |

The attacker sits on the customer edge, on a point-to-point link to the router's `cust` interface. The victim and the two reflectors share the core segment behind the router's `core` interface, on one Open vSwitch switch. The attacker's NIC is 102-cust; every core host's NIC is 102-S1.

The attacker carries stock tooling; there is no bespoke exploit:

- **nping** (from the nmap project): sends UDP or ICMP with a source address you choose (`-S`). This is how you forge a source.
- **scapy** (Python): builds the reflected DNS and NTP queries field by field, including the forged source. You type every field, so you see exactly what leaves.
- **reflect**: a baked one-command version of the same two queries. It takes no default addresses; you still name the reflector, the protocol, and the source you forge.
- **tcpdump**, **dig**: read what arrives and measure how big it is.

The router forwards between the two subnets and does no source checking yet. That gap is the attack surface; closing it is Part 2. Open a shell wherever a step says so with:

```
docker exec -it <container> bash
```

**Assessment.** This lab is graded on eighteen questions, marked inline below as **Question *N***, each placed at the step whose output it asks about. Answer all eighteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step.

## Part 1: the attack

### Honest traffic first

#### Definition: TXT record

A DNS record type that holds arbitrary text rather than an address. Domains publish TXT records for policy and verification data; one name can carry many TXT strings, and a query for that name returns them all in a single reply.

1. **Send one query the honest way.** On the attacker, ask the DNS reflector for its one record, using your own address as the source:

```
docker exec -it 102_L3_LAB_attacker bash
dig @102.1.0.53 amplify.lab TXT           # a 250-byte TXT record comes back
```

The reply reaches you because the query carried your real source, 102.0.0.10, so the reflector returns the **TXT record** to you. Note the **MSG SIZE rcvd** line; you will compare it to the request shortly.

**Question 1.** You sent this query from your own address and the reply came back to you. Predict what changes about where the reply goes if the source field named the victim (102.1.0.20) instead, and explain what makes UDP reflection possible that a TCP-based service would not allow.

### Forge a source

2. **Send a packet from an address that is not yours.** Still on the attacker, aim UDP at the victim but claim a source on no lab subnet (203.0.113.66, TEST-NET-3). In a second shell, watch the victim receive it:

```
docker exec -it 102_L3_LAB_victim bash
tcpdump -n -i 102-S1 'src host 203.0.113.66'      # leave running
```

```
# back on the attacker:
nping --udp -S 203.0.113.66 -p 9 -c 4 --rate 5 -e 102-cust 102.1.0.20
```

The victim's `tcpdump` shows four packets from 203.0.113.66. The router forwarded them from `cust` to `core` without asking whether that source could legitimately be there.

**Question 2.** The victim received four packets sourced from 203.0.113.66, an address no host owns. State what the router did with each packet and which check it did not perform.

**Question 3.** Any reply the victim sends goes to 203.0.113.66, not to you. Name one thing forging the source already buys the attacker, before any reflection is involved.

## Reflect off DNS

3. **Point a reflector's reply at the victim.** Now forge the source as the *victim* and query the DNS reflector. The reflector will answer the victim, which never asked. Start the capture *first* and leave it running; the network is otherwise idle, so what it catches is the attack:

```
# on the victim:
tcpdump -ne -i 102-S1 'src host 102.1.0.53 and dst host 102.1.0.20'
```

Then, on the attacker, build the query by hand in scapy so every field is yours, forging the victim as the source:

```
# on the attacker:
python3 -c "from scapy.all import *; \
send(IP(src='102.1.0.20', dst='102.1.0.53')/UDP(dport=53)/ \
    DNS(rd=1, qd=DNSQR(qname='amplify.lab', qtype='TXT')), count=4,
    inter=0.2)"
```

The baked tool sends the same packet: `reflect dns 102.1.0.53 102.1.0.20 4`. On the victim's capture, four replies arrive from the DNS server, each carrying the 250-byte TXT record.

**Question 4.** The query's source was set to 102.1.0.20. Explain why the victim, not the attacker, is the one flooded.

**Question 5.** Compare the request size to the reply size. Take the request as roughly the `nping/dig` query on the wire (about 55–70 bytes) and read the reply length from `tcpdump`. State the approximate amplification factor and how you got it.

**Question 6.** The record queried is a single 250-byte TXT string. Explain why the reply size, not the request size, sets the amplification factor, and what an attacker looks for in a name to query.

## Real reflectors: which domain amplifies most

4. **Measure real domains, not just the lab record.** A real attacker does not invent a record; it looks for names already published on the internet whose replies are large. This reflector also serves the TXT record sets of three well-known domains, copied verbatim from `dig TXT` on 2026-07-28: `google.com` carries 15 TXT records, `spotify.com` 35, and `amazon.com` 61. These are the real published TXT records from those sites on that date. From the attacker, using your own source so each reply comes back to you, measure the three:

```
# on the attacker, one line per domain:
dig +bufsize=4096 +ignore @102.1.0.53 google.com TXT | grep 'MSG SIZE'
dig +bufsize=4096 +ignore @102.1.0.53 spotify.com TXT | grep 'MSG SIZE'
dig +bufsize=4096 +ignore @102.1.0.53 amazon.com TXT | grep 'MSG SIZE'
```

### Definition: EDNS0

Extension mechanisms for DNS. A plain DNS query tells the server the client accepts at most 512 bytes over UDP, so any larger answer is truncated. An EDNS0 option in the query advertises a bigger buffer (here up to 4096 bytes), so the server may return the larger answer over UDP.

`+bufsize=4096` advertises a 4096-byte **EDNS0** buffer, so the reflector may answer over UDP with more than the default 512 bytes. `+ignore` stops `dig` retrying over TCP when the reply is truncated, so the size you read is what one spoofed UDP query could reflect. `google.com` returns about 1050 bytes and `spotify.com` about 2800, both under the 4096 ceiling. `amazon.com`'s set does not fit: the reply comes back with the `tc` (truncated) flag set, capped at the 4096-byte ceiling. Drop `+ignore` and query `amazon.com` again, and `dig` retries over TCP and reports the full set at about 5100 bytes. Then shrink the buffer to see the record size is not what limits the UDP reply:

```
# on the attacker: a small buffer truncates the same record
dig +bufsize=512 +ignore @102.1.0.53 amazon.com TXT | grep -E 'MSG SIZE|flags'
```

The reply is `tc`-flagged and about 512 bytes, the same truncation the `reflect` tool's plain query hits, because that query advertises no larger buffer. The buffer the query asks for, not the size of the record, sets how much comes back over UDP.

**Question 7.** Record the `MSG SIZE rcvd` for all three domains. Compute the approximate amplification factor for each (reply size divided by the small query, on the order of 40 to 60 bytes), and rank the three domains by that factor.

### Definition: SPF record

A TXT record beginning `v=spf1` that lists which hosts may send email for a domain. Mail receivers read it to tell legitimate senders from forgeries.

Rerun the three queries without `grep` and read a few strings of each. Most are not addresses: one is the domain's **SPF record**, and the rest are ownership tokens that third-party services (Facebook, Atlassian, Stripe, and others) ask a domain to publish to prove it controls the domain.

**Question 8.** A large company's TXT set runs to dozens of these tokens where a small domain's is a handful. Explain why the set grows the way it does over a company's lifetime, and what that pattern tells an attacker choosing a name to reflect off.

**Question 9.** The `reflect` tool and the `scapy` query in step 3 advertise no large buffer, so a big answer comes back truncated to 512 bytes. Explain why a forged query has to advertise a large EDNS buffer to draw the amplified reply you measured, and what `amazon.com`'s truncation near 4096 bytes shows about the ceiling on UDP reflection, given that a spoofing attacker cannot fall back to TCP.

## Reflect off NTP

### Definition: NTP control query (mode 6)

A management request to an NTP server, separate from ordinary time synchronisation. A mode-6 `readvar` asks the server to return its whole internal variable list in one reply.

5. **Use a second, unrelated service.** NTP's original amplifier was the mode-7 `monlist` command, removed from servers after it was abused in 2014. This lab uses the vector that replaced it: a mode-6 `readvar` **control query**, which returns the server's whole variable list. Twelve bytes of request draw a reply several hundred bytes long. Start the capture on the victim *first*:

```
# on the victim:
tcpdump -ne -i 102-S1 'src host 102.1.0.123 and dst host 102.1.0.20'
```

Then fire the query from the attacker, forging the source as the victim again:

```
# on the attacker:
python3 -c "from scapy.all import *; \
    send(IP(src='102.1.0.20', dst='102.1.0.123')/UDP(dport=123)/ \
        Raw(b'\x16\x02'+b'\x00'*10), count=4, inter=0.2)"
```

The baked equivalent is `reflect ntp 102.1.0.123 102.1.0.20 4`. The twelve bytes `16 02 00 ...` are exactly what `ntpq -c rv` puts on the wire: version 2, mode 6 (control), opcode 2 (read variables). A byte too many or too few and the server treats it as malformed and returns almost nothing.

**Question 10.** Measure the NTP reply length on the victim and compare it to the three real DNS replies you measured in step 4 (`google.com`, `spotify.com`, `amazon.com`). State which service amplifies more here, and by roughly how much.

## Why bounce it at all

6. **See the attacker's position.** At the victim, the flood arrives from `102.1.0.53` and `102.1.0.123`: two legitimate servers. The attacker's own address, `102.0.0.10`, appears nowhere in what the victim sees.

**Question 11.** Give two reasons an attacker prefers bouncing traffic off reflectors over sending it straight to the victim. Tie each reason to something you observed in steps 3 to 5.

**Question 12.** The victim sees the flood coming from two real servers it may depend on. Explain why blocking those source addresses at the victim is not a usable fix.

## Part 2: the defence

The attack works only because the router forwards a packet without asking whether its source address belongs where it arrived. The defence closes exactly that gap, and it goes on the router at the *attacker's* edge, not near the victim: the network a packet leaves is the only one that can tell the source is forged, because only it knows which addresses live behind that interface. Both mechanisms below enforce the same one rule at the customer interface.

### Definition: Source-address verification

Checking, at the interface a packet arrives on, that its source address is one that could legitimately originate there, and dropping it otherwise. It is applied at the edge closest to the sender, not near the target.

**Definition: BCP 38 (ingress filtering)**

The IETF best-practice document that tells a network to drop, at its customer edge, any packet whose source address does not belong to that customer. It is the standard behind source-address verification.

The victim is drowning, but the fix does not go near the victim. It goes on the router at the *attacker's* edge, on the `cust` interface, and it is one idea: a packet arriving on an interface must carry a source address that could legitimately come from that interface. This is **source-address verification**, the **ingress filtering** of **BCP 38**. Try either mechanism below; both run on the router.

**Question 13.** The flood lands on the victim, yet the filter belongs on the router beside the attacker. Explain why a filter at the victim's edge cannot stop a reflection attack, and what the source-side filter stops that the victim-side one cannot.

**Option A: strict uRPF on the router****Definition: uRPF (unicast reverse-path forwarding)**

A per-packet check that consults the routing table: for a packet's source address, would the route back to it leave by the interface the packet just arrived on? In strict mode, if the answer is no, the packet is dropped.

**Unicast Reverse Path Forwarding** (uRPF) asks the routing table a question for every incoming packet: if I had to reach this packet's *source*, would I send it back out the interface it just arrived on? If not, drop it. Turn it on for the customer interface (the router runs privileged, so `/proc/sys` is writable here):

```
docker exec -it 102_L3_LAB_router bash
sysctl -w net.ipv4.conf.all.rp_filter=0      # keep 'all' off; enforce per-interface
sysctl -w net.ipv4.conf.cust.rp_filter=1    # 1 = strict
```

Re-run every attack from Part 1. The forged-source packets stop arriving: the victim's `tcpdump` goes quiet, and the reflectors receive nothing to reflect. Confirm honest traffic is untouched by running the step-1 `dig` again; it still answers.

**Question 14.** Apply strict uRPF to the step-3 forged packet: source 102.1.0.20 arriving on `cust`. Say which route lookup uRPF performs on it and why that lookup makes the packet fail.

**Question 15.** `rp_filter=1` is strict; `rp_filter=2` is loose (the source need only be reachable by *some* route, not necessarily out the ingress interface). Describe one legitimate topology where strict mode would wrongly drop real traffic and loose mode would not.

**Option B: an nftables ingress ACL****Definition: ACL (access control list)**

An ordered list of match-and-action rules a device applies to packets. Here it permits sources inside the customer subnet and drops the rest, naming the allowed range literally rather than deriving it from the routing table as uRPF does.

An **ACL** states the same policy without consulting the routing table: on `cust`, permit only sources inside the customer subnet, drop the rest.

First undo Option A (`sysctl -w net.ipv4.conf.cust.rp_filter=0`), then, on the router:

```
nft -f - <<'EOF'
table ip bcp38 {
    chain ingress {
        type filter hook prerouting priority -300; policy accept;
        iif "cust" ip saddr != 102.0.0.0/24 counter drop
    }
}
EOF
```

Re-run the attacks; they are dropped again. This time you can read how many packets the rule caught:

```
nft list table ip bcp38          # the drop rule's counter climbs with each forged
                                packet
```

Reference solution: `solution/nftables_acl.sh`.

**Question 16.** The rule drops when `iif "cust"` and `ip saddr != 102.0.0.0/24`. Explain why the attacker's honest traffic (source 102.0.0.10) passes while every forged source you sent is dropped.

**Question 17.** uRPF reads the routing table; the ACL names the subnet literally. Give one operational advantage of each: what uRPF handles for free that the ACL must be told, and what the ACL makes explicit that uRPF leaves implicit.

### What this fixes, and its limit

**Question 18.** Suppose every operator ran this filter at its own customer edge. Of the two attack stages here, direct spoofing and reflection, state which stop working and why. Then explain why one network skipping the filter still leaves victims elsewhere exposed.

## Checks: what “done” means

- **Attack works:** with no filter, the victim's `tcpdump` shows forged-source packets and reflected replies from 102.1.0.53 and 102.1.0.123, each reply larger than the query that triggered it.
- **Defence holds:** with uRPF or the ACL on `cust`, the forged-source packets never leave the router, the victim's capture is empty, and the attacker's honest `dig` still answers.

`scripts/status.sh` fires one forged query at each reflector and reports REACHES or BLOCKED, so you can read before and after without shelling in. To undo your work, `scripts/reset.sh` stops the attack and drops the filter, back to the Part-1 baseline. `scripts/teardown.sh` removes the lab.

**Safety.** Everything here lives on one isolated router and switch with no route off the host. The forged packets and the reflected replies only ever reach the lab's own containers.

**When you are done, tear the lab down.** Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left



running on your host. **Reset to baseline** only rewinds the attack and the defence; only **Teardown** deletes the lab.